Planning and Reinforcement Learning in AI Systems

(PARLAI)

Lesson 5a - Planning and Learning

Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

Planning and Learning

The Curse of Dimensionality

Many control algorithms scale poorly with the number of states.

Dynamic programming requires operations proportional to $O(|\mathcal{A}||\mathcal{S}|^2)$

In realistic domains:

- Robotics: $n \approx 10^{12}$ or more
- Games: enormous state spaces
- Real-world systems: continuous states

Exact planning quickly becomes impossible.

This motivates **approximation and learning**.

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

What Is Planning?

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

What Is Planning?

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Definition

Planning is the process of computing a sequence of actions or a policy that achieves goals or maximizes expected utility, given an explicit model of the environment.

Input:

- State space $\mathcal{S}$ and action space $\mathcal{A}$
- Transition model $\mathcal{P}(s' \mid s, a)$
- Reward / cost function $\mathcal{R}(s, a)$
- (Optional) observation model $\mathcal{O}(o \mid s)$

What Is Learning?

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Definition

Learning is the process of improving performance or knowledge from data or experience by updating internal models, representations, or policies.

What can be learned?

What Is Learning?

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Definition

Learning is the process of improving performance or knowledge from data or experience by updating internal models, representations, or policies.

What can be learned?

- Dynamics model $\mathcal{P}(s' \mid s, a)$
- Observation model $\mathcal{O}(o \mid s)$
- Reward model $|\mathcal{R}(s, a)$
- Policy $\pi(a \mid s)$ or value function $V(s)$

Planning vs. Learning

Planning

- Model-based
- Deliberative (lookahead)
- Uses prior knowledge
- Output: plan or policy

Learning

- Data-driven
- Adaptive
- Reduces modeling errors
- Output: model or policy

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Planning vs. Learning

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Planning

- Model-based
- Deliberative (lookahead)
- Uses prior knowledge
- Output: plan or policy

Learning

- Data-driven
- Adaptive
- Reduces modeling errors
- Output: model or policy

Learning provides models or policies; planning reasons over them.

What do we need to do each of these efficiently? When and how should we combine them ?

Why Planning and Learning ?

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Real-world and physical AI-systems involve:

- High-dimensional continuous spaces
- Noisy and partial observations
- Safety and physical constraints

Learning alone is brittle. Planning alone is unrealistic.

Example: Planning with Learned Perception

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation



Learning:

- Vision model predicts object pose probabilities
- Slip / no-slip classifier from force sensors

Planning:

- Maintain belief over object states
- Select actions robust to uncertainty
- Decide when to re-sense or regrasp

Why Combine Planning and Learning?

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Why Combine Planning and Learning?

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Planning:

- Explicit reasoning over future actions
- Structured decision-making with constraints

Learning:

- Handles high-dimensional sensory inputs
- Adapts from data and experience

Kitchen robot example:

- Learning detects mugs, kettle, coffee jar from images
- Planning decides the order: fetch mug → boil water → pour

Bridging the Gap

Model-based planning: **accurate but brittle**
Learning-based methods: **adaptive but less predictable**

Key question:

Can we get the best of both worlds?

Complementary strengths:

- Planning:
  - strong guarantees (optimality, safety)
  - interpretable structure
- Learning:
  - adapts to unknown or changing environments
  - scales to high-dimensional problems

Bridging the Gap

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Emerging direction:

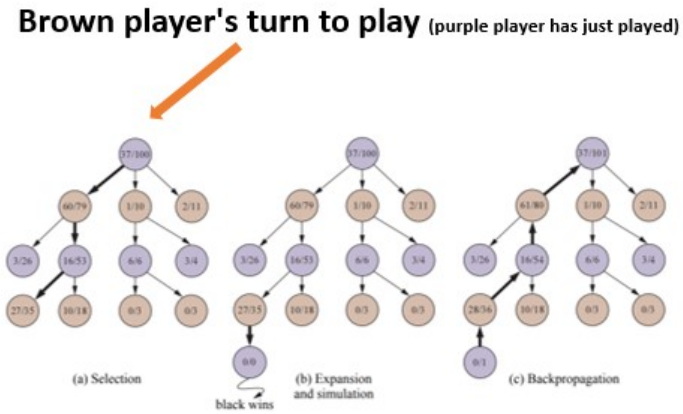
- Combine **model-based reasoning** with **data-driven learning**
- examples:
  - model-based RL
  - learned heuristics for planning
  - hybrid planning + learning systems

Modern AI systems integrate planning and learning rather than choosing one

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Monte Carlo Tree Search (MCTS)

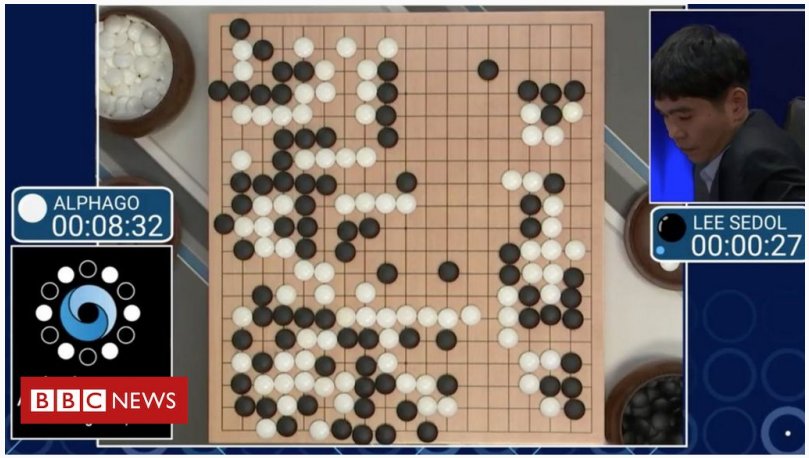
- **Simulation policy**:
  - Use random moves (pure Monte Carlo)
  - Learn / use expert knowledge
- **What is a good selection policy?**
  - At the source node it's the brown player's turn to plan – 60/79 seems the most promising node
  - But maybe 2/11 is better but we haven't tried it enough



From IAI

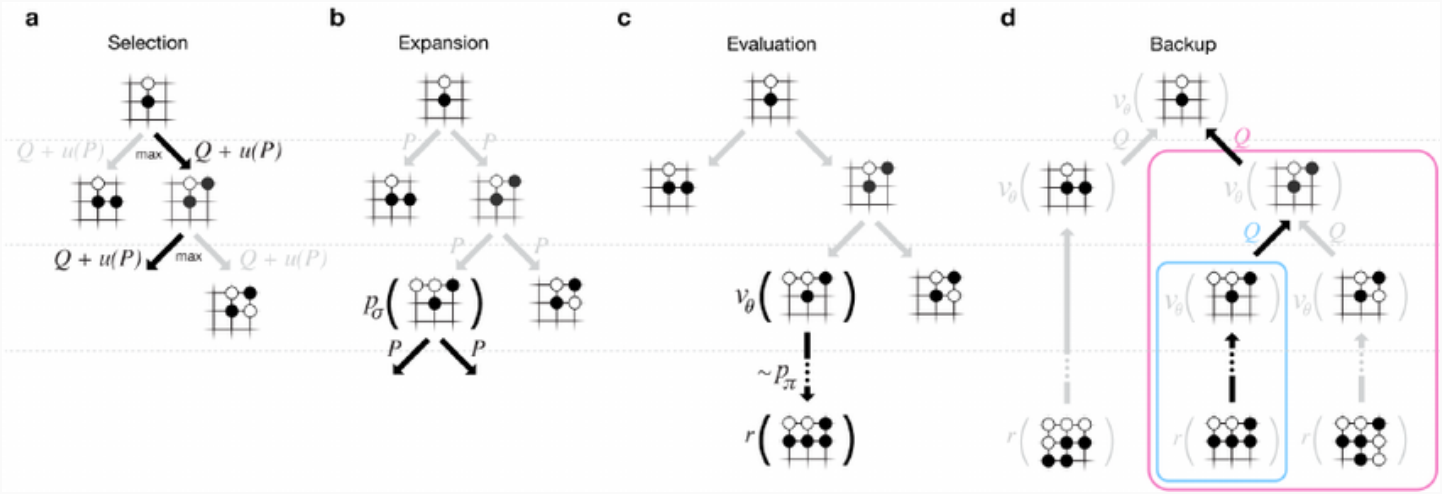
Monte Carlo Tree Search

- Combines exploration and exploitation to choose an action in a tree search setting
- Relevant for a variety of problems: classical planning, stochastic planning, and game playing (breakthrough via the game of Go)
- Many different variants
- Applications far beyond games: Planning, motion planning, optimization, finance, energy management



Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Example: MCTS for GO



Planning and Learning

Selection

Simulation

MCTS

MCTS Evaluation

MCTS is not just for games

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

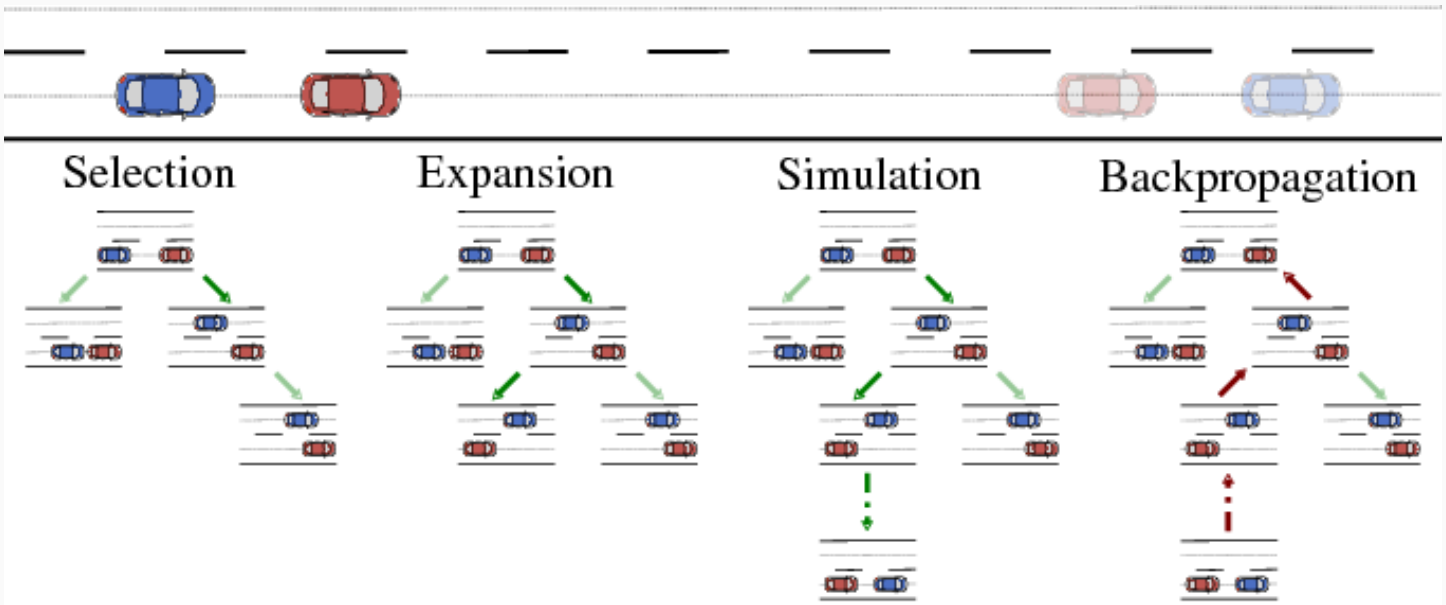


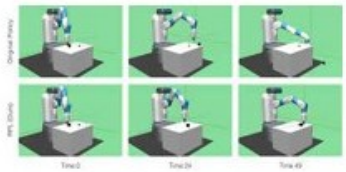
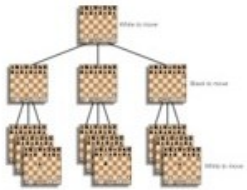
Fig. 1: Phases of Monte Carlo Tree Search for an overtaking maneuver; the

https://www.semanticscholar.org/paper/Decentralized-Cooperative-Planning-for-Automated-Kurzer-Zhou/585b73322365ba2d0afa7449691c81cb98777599

Simulation (Rollout)

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

- A simulation (also called playout or rollout) chooses moves according to some policy, until reaching a terminal state
- Various options for simulation policy (e.g., random)
- The value of a state is estimated as the average utility over a number of simulations


Used in many applications and in different algorithms.

Generative Models

In some settings, we can collect data from the environment of interest. In others, we have access to a process that can generate possible outcomes.

For example, a simulator may allow us to query:

$$(s', r) \sim G(s, a)$$

and observe the consequences of actions.

Such a **generative model** can be used to simulate trajectories and evaluate decisions through lookahead search.

A simulator can be used to generate training data for model-free learning, but access to the simulator itself enables model-based decision making.

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Simulators



Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

How to use simulations for decision making?

Naïve Approach

- Run many rollouts from the root node
- Use rollouts directly as an evaluation function
- Problems:
  - Single rollout is very noisy, only 0/1 signal
  - Running many simulations for one evaluation is very slow
  - In some cases, running a single rollout is costly

Performance guarantees? Limitations?

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Monte Carlo Tree Search

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Approximate optimal decision-making in large search spaces by *sampling* rather than exhaustive enumeration.

Core idea:

- Build a search tree incrementally
- Use random simulations (rollouts) to estimate action values
- Focus computation on promising parts of the tree

Key Assumption:

Monte Carlo Tree Search

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Approximate optimal decision-making in large search spaces by *sampling* rather than exhaustive enumeration.

Core idea:

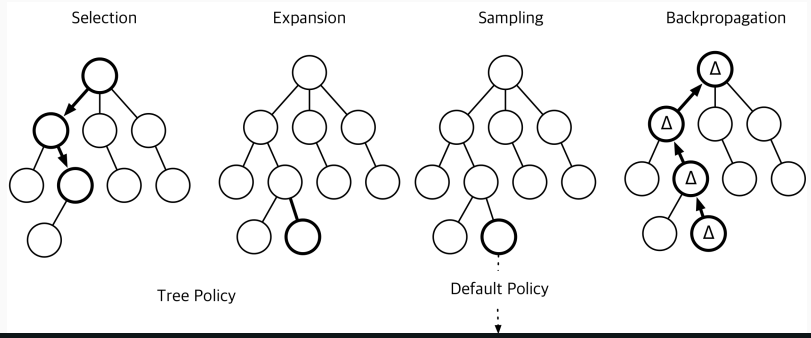
- Build a search tree incrementally
- Use random simulations (rollouts) to estimate action values
- Focus computation on promising parts of the tree

Key Assumption: An efficient simulator is available.

Monte Carlo Tree Search

Key Idea: use **Monte Carlo simulation** to accumulate value estimates to guide towards highly rewarding trajectories in a **search tree**.

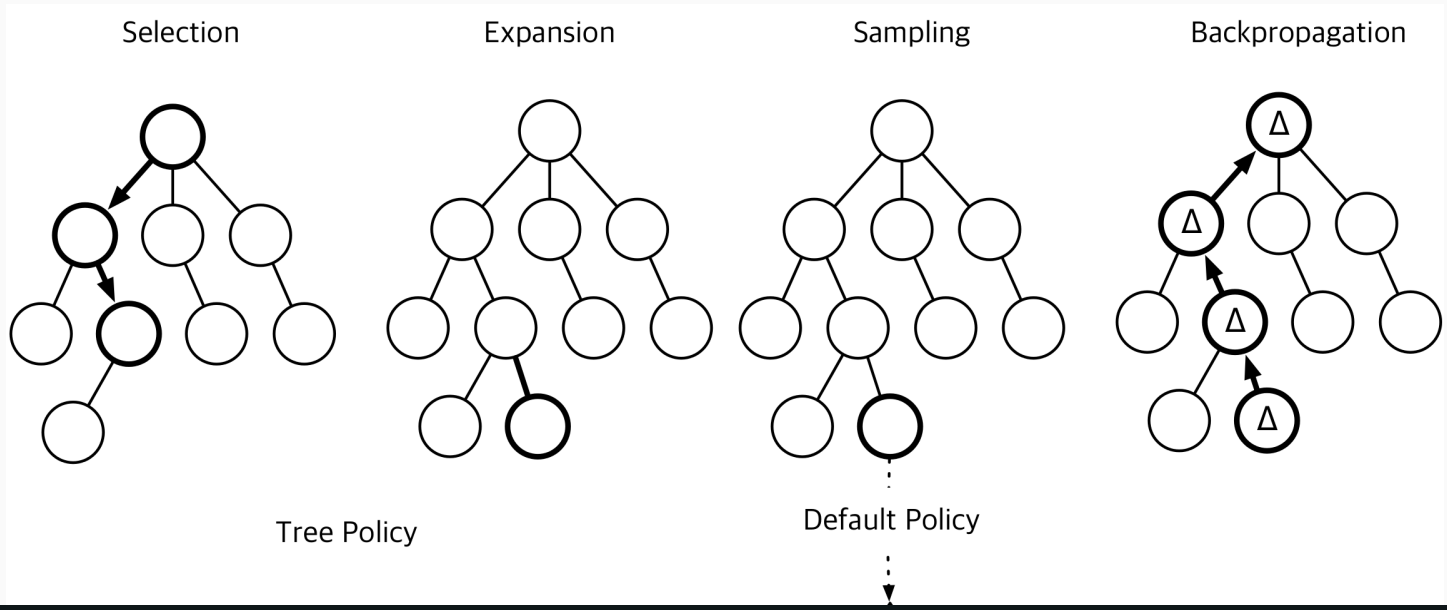
- Each simulation consists of two phases
  - **Tree policy:** pick actions to maximise values
  - **Default / roll-out policy:** pick actions randomly to simulate a trajectory.
- Repeat (each simulation)
  - **Evaluate** states $Q(S, A)$ by Monte-Carlo evaluation
  - **Improve** tree policy e.g. by ϵ-greedy.
- Converges on the optimal search tree $Q(S, A) \to q(S, A)$



Monte Carlo Tree Search

From IAI:

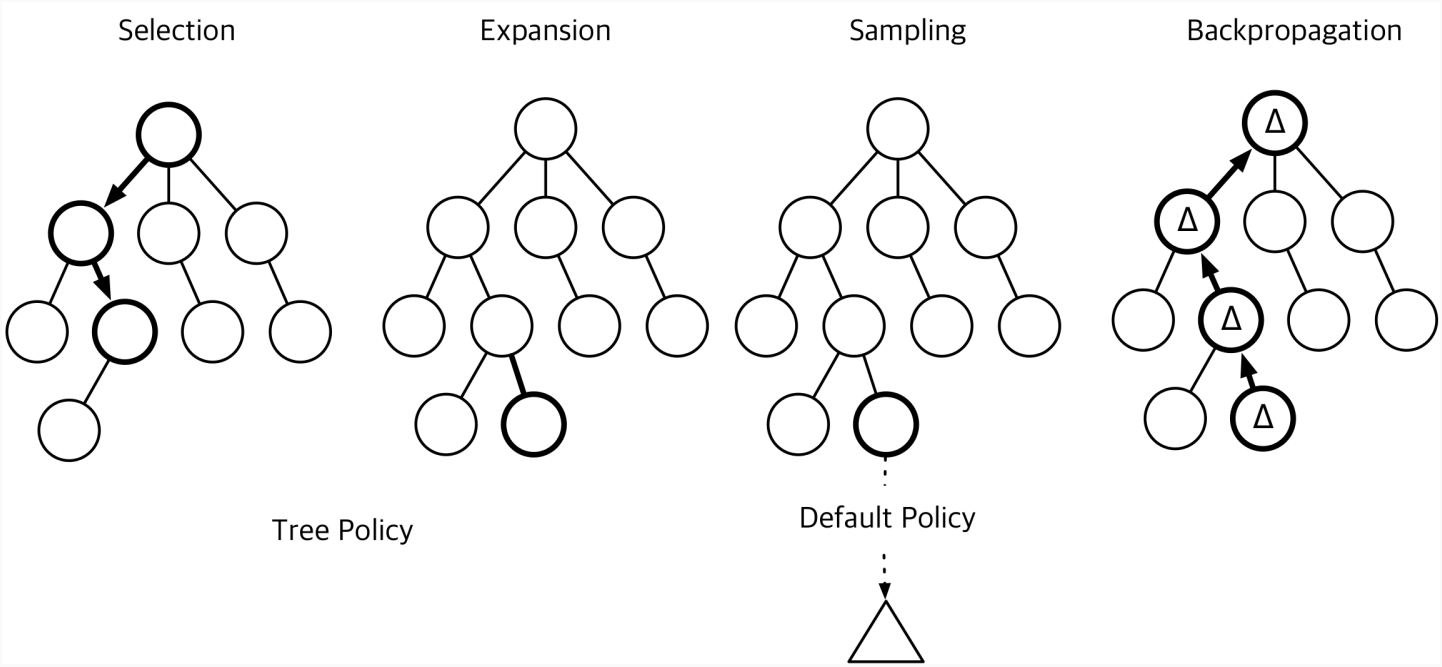
function MONTE-CARLO-TREE-SEARCH(*state*) **returns** *an action*
  tree ← NODE(*state*)
  while IS-TIME-REMAINING() **do**
    leaf ← SELECT(*tree*)
    child ← EXPAND(*leaf*)
    result ← SIMULATE(*child*)
    BACK-PROPAGATE(*result*, *child*)
  return the move in ACTIONS(*state*) whose node has highest number of playouts



Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Monte Carlo Tree Search

- Selection
- Expansion
- Sampling / Simulation
- Back-propogation



Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Selection

Selection Policy

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

In MCTS we Use results of simulations to guide growth of the search tree

- **Exploitation:** focus on promising moves
- **Exploration:** focus on moves where uncertainty about evaluation is high

Two contradictory goals: how to manage the tradeoff ?

Exploration–Exploitation Tradeoff

At any decision point, an agent faces two competing objectives:

$$\text{Exploit} \iff \text{Maximize immediate reward}$$

$$\text{Explore} \iff \text{Improve future decisions}$$

Exploration may reduce short-term performance, but insufficient exploration may prevent discovering better actions.

How much reward should we sacrifice now in order to make better decisions later?

Exploration, Exploitation, and Regret

Every exploratory action carries a potential cost.

Choosing an uncertain action instead of the currently best-known action may lead to a lower immediate reward.

Regret quantifies the **cost of learning, exploration, or imperfect decision-making**: it measures the performance loss incurred by an agent due to not always making the best decisions.

$$R(T) = \underbrace{\text{Cost of exploration}}_{\text{information gathering}} + \underbrace{\text{Cost of wrong estimates}}_{\text{insufficient learning}}$$

The goal is not to avoid exploration, but to explore efficiently so that regret grows as slowly as possible.

Regret

Let $\pi^\star$ denote an optimal policy and π the policy executed by the agent. Regret after T steps is the difference between the expected return of π^* and that of π:

$$R_T = V_T^{\pi^\star}(s_0) - V_T^{\pi}(s_0)$$

What are possible formulations of V?

- **Zero regret**: the agent performs as well as the optimal policy.
- **Positive regret**: the agent loses reward due to suboptimal actions.
- **Sublinear regret** (e.g., $O(\sqrt{T})$): average regret per step converges to zero.

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Selection Policies

How should an agent choose among actions whose values are not yet known? How can we design a selection policy that achieves low regret?

Common approaches include:

- **Greedy:** always select the currently best action
- **ϵ-Greedy:** occasionally explore random actions
- **Thompson Sampling:** sample actions according to their probability of being optimal
- **Upper Confidence Bounds (UCB):** prefer actions with high value or high uncertainty

A classical setting for studying this question is the **Multi-Armed Bandit Problem**, which leads to the **Upper Confidence Bound (UCB)** selection rule.

Selection Policy: Multi-Armed Bandit Problem

- We can choose among several arms
- Each arm pull is independent of other pulls
- Each arm has fixed, unknown average payoff
- Which arm has the best average payoff?







Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Selection Policy: Multi-Armed Bandit Problem



Which arm to pull ?

Assume payoff is 1. Which arm to pull if we pulled each arm twice and A was the only arm that yielded value ?

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Selection Policy: UCB1 [Auer et al 02]

- First, try each arm once
- Each arm pull is independent of other pulls
- Then, at each time step:
- Choose arm i that maximizes the UCB1 formula for the upper confidence

$$v_i + C \times \sqrt{\frac{ln(N)}{n_i}}$$

where

- v_i - current estimation of the value of bandit i
- C - tunable parmater to balance exploration / exploitation
- N total number of trials
- n_i - no. of trials for bandit

How is this relevant to search trees ?

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Selection Policy: UCT (UCB applied to trees)

- UCB makes single decision
- What about sequences of decisions

Selection Policy: UCT (UCB applied to trees)

- UCB makes single decision
- What about sequences of decisions
- Apply UCB-like formula for node selection
  - Choose ”optimistically” where to expand next

$$UCB(S) = \frac{U(S)}{N(S)} + C \times \sqrt{\frac{ln(N(S.parent))}{N(S)}}$$

where

- $U(S)$ - total utility of all rollouts that went through S
- N(S) - number of rollouts though S.

Simulation

Simulation Policy

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

After a node is expanded, MCTS must estimate its value.

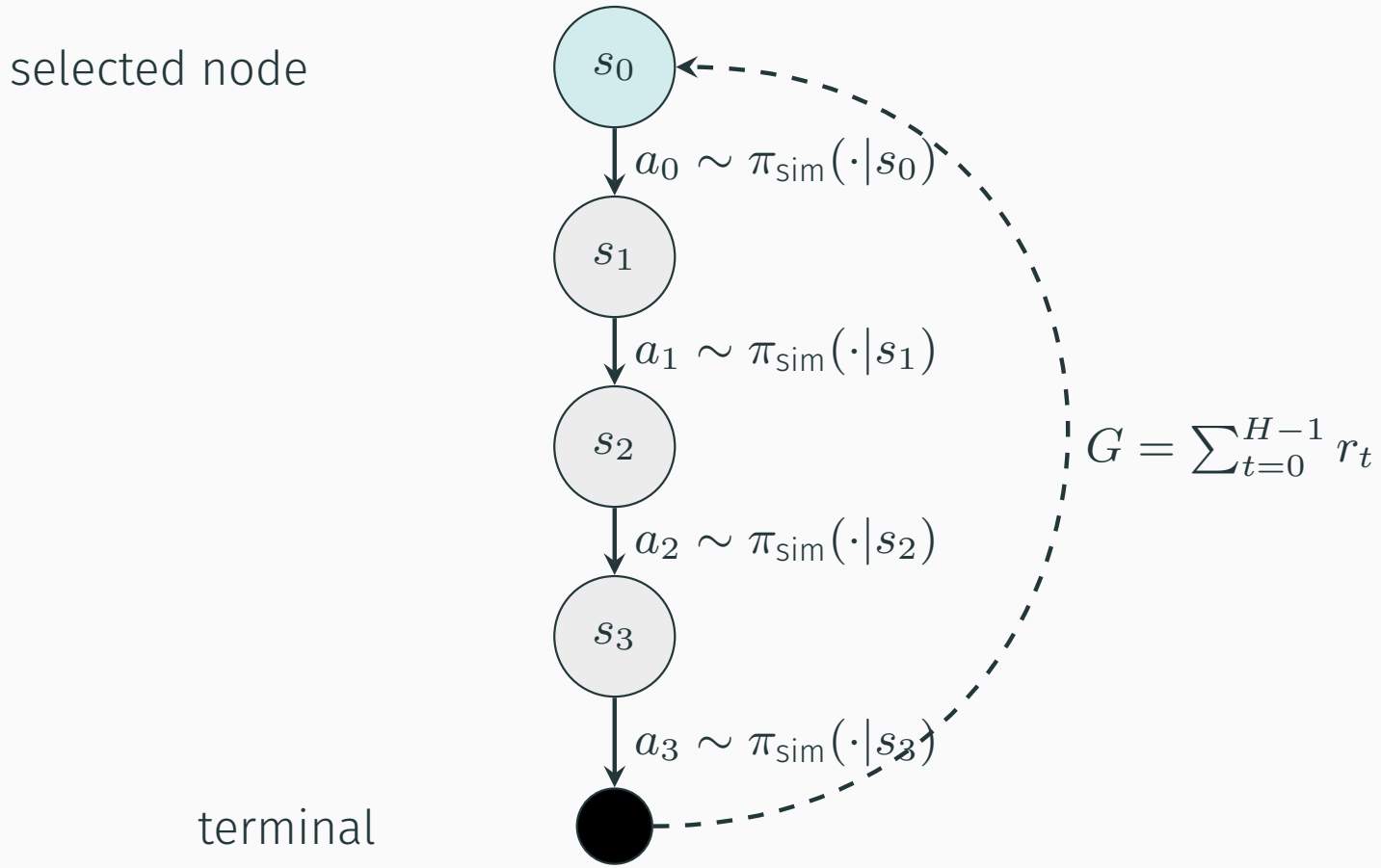
- **Selection policy:** decides which node to explore next.
- **Simulation policy:** estimates the value of the selected node by simulating the remainder of the episode.

The simulation proceeds until a terminal state (or a depth limit) is reached and returns an outcome.

For example, in a game win, loss, draw. or more generally, a cumulative reward **return** $= \sum_{t=1}^{T} r_t$.

How should actions be selected during the simulation?

Simulation / Roll-out Policy



How should actions be selected during the simulation?

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Simulation / Roll-out Policy

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Possible approaches include:

- **Random rollouts:** choose actions uniformly at random.
- **Rule-based rollouts:** use simple domain-independent heuristics (e.g., take immediate wins, avoid immediate losses).
- **Knowledge-based rollouts:** incorporate domain knowledge or handcrafted evaluation rules.
- **Learned rollouts:** use a learned policy to guide simulations.
- **Value-based evaluation:** replace long simulations with a value function estimate.

MCTS

Monte Carlo Tree Search

function MCTS(s_0, n)
Require: Initial state s_0, number of simulations n
    for $i = 1$ to n **do**
        MCTSTRIAL(s_0)
    end for
    return $\arg\max_a Q(s_0, a)$
end function

function MCTSTRIAL(s)
    if s is terminal **then**
        return $r(s)$
    end if
    if s is not in the tree **then**
        Add s to the tree
        return ROLLOUT(s)
    end if
    Select action a using the tree policy
    $s' \leftarrow$ NEXTSTATE(s, a)
    $R \leftarrow r(s, a, s') + \gamma$MCTSTRIAL($s'$)
    Update Q and visit counts
    return R
end function

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

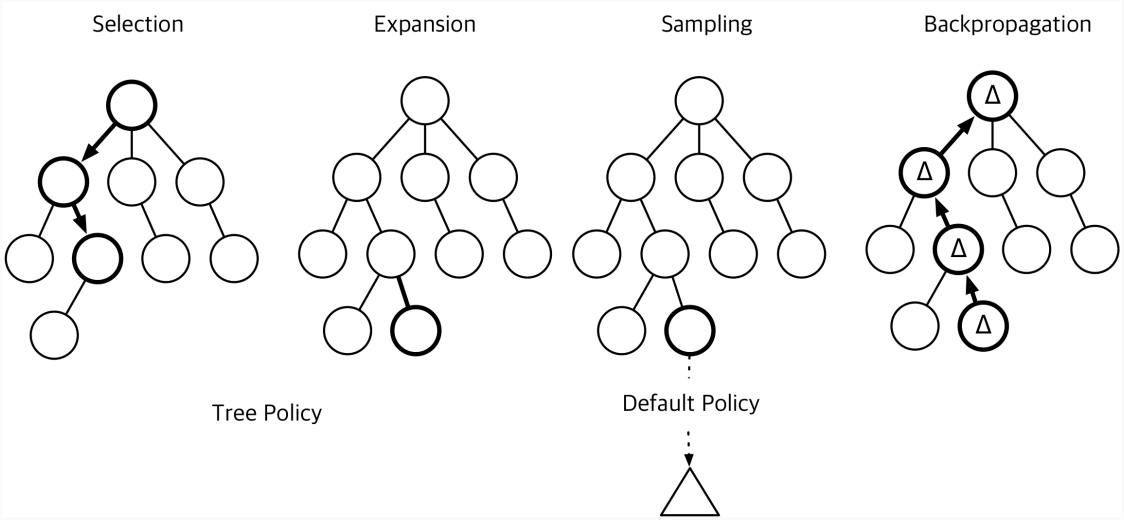
MCTS efficiency

- The time to perform a rollout is linear in the depth of the tree
- This gives plenty of time to consider multiple rollouts
- For example
  - if:
    - Branching factor b=32
    - Average game length (tree depth) is d=100
    - We can compute 10^9 moves
  - Minimax can search 6 ply deep
  - AlphaBeta can search up to 12 ply deep
  - MCTS can do 10^7 rollouts
- Works great for games with
  - Large branching factor (then minimax can't search deep enough)
  - Poor evaluation function

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

The Generality of MCTS

- Very general algorithm for decision making
  - Works with very little domain-specific knowledge
  - (But) needs a simulator of the domain
  - Can take advantage of knowledge when present
  - Anytime algorithm - can stop the algorithm and provide an answer immediately, though it improves the answer with more time



Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

The Generality of MCTS

Monte Carlo Tree Search is a **domain-agnostic planner**.

Minimal assumptions:

- A state (or belief) representation
- An action set
- A **generative model** to simulate outcomes
- A reward signal

Strengths:

- No explicit transition or observation matrices required
- Works with black-box simulators or learned models
- Same algorithm applies across domains

Used in games (Go, Chess), robot manipulation and navigation, and planning under uncertainty (MDPs and POMDPs).

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

MCTS as a Planner

Core idea: Use MCTS to select actions by reasoning over *future outcomes* using simulations.

Planning perspective:

- **State** (or belief): current situation of the robot/world
- **Actions**: choices available to the agent
- **Model**: simulator or generative world model
- **Objective**: maximize expected cumulative reward

Key distinction from classical planners:

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

MCTS as a Planner

Core idea: Use MCTS to select actions by reasoning over *future outcomes* using simulations.

Planning perspective:

- **State** (or belief): current situation of the robot/world
- **Actions**: choices available to the agent
- **Model**: simulator or generative world model
- **Objective**: maximize expected cumulative reward

Key distinction from classical planners:

- No full state-space enumeration
- No explicit value-function backup over all states
- Planning is **anytime** and **online**

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

MCTS Evaluation

Evaluating Planning Algorithms

Dimensions for evaluation

- **Completeness**: always find a solution if one exists?
- **Time complexity**: number of nodes generated/expanded
- **Space complexity**: number of nodes in memory
- **Optimality**: does it always find a least-cost solution?
- **Anytime**: does the solution improve the more resources are used ?

Time/space complexity measured in terms of

b maximum branching factor of the search tree

d depth of the least-cost solution

m maximum depth of the state space (may be ∞)

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Evaluating Algorithms: Modern Considerations

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

Robustness: how sensitive is performance to noise, uncertainty, or modeling errors?

Scalability: how does performance change as the problem size grows?

Adaptivity: can the algorithm improve or adjust based on experience?

Generality: does the method apply broadly across domains or rely on strong assumptions?

Evaluating Algorithms: Modern Considerations

Planning and Learning
Selection
Simulation
MCTS
MCTS Evaluation

No exploration vs. exploitation since the model is assumed to be known.

No account for **uncertainty** or **risk** since the model includes deterministic actions.

Interpretability is inherent, since the model is given.

No explicit account for **energy efficiency** beyond standard analysis of worst-case complexity.

Strengths and Limitations of MCTS

Strengths

Limitations

Strengths and Limitations of MCTS

Strengths

- **Domain-agnostic**: works with any generative model
- **Anytime planning**: can be stopped at any time
- **Scales to large spaces**: focuses computation on promising actions
- **Naturally stochastic**: handles uncertainty without discretization

Limitations

- **Simulator dependence**: performance tied to model fidelity
- **Computationally intensive**: many rollouts per decision
- **Weak short-horizon reasoning**: shallow rollouts can be misleading
- **Poor with sparse rewards**: hard to guide exploration